

From: <https://chatgpt.com/c/6a8a279f-3c40-83ee-8aa1-24574734f5ea>

you asked: go through

message time: 2026-08-23 04:17:53

go through the previous chats from start to end completely which are attached files and responses how detailed- <https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93> , <https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5> , <https://chatgpt.com/c/6a8a20f0-f3fc-83ee-891b-ba811975393f> , <https://chatgpt.com/c/6a8a2356-f96c-83ee-950e-82f71a5e00eb> , <https://chatgpt.com/c/6a8a251a-31f8-83e8-9107-c2dfbcaab421>

And continue further according to previous - Module 1,2,3,4,5 completed as above their links given and also attached files . Now continue with **Module 6:** 75 Spring Security/JWT questions.

For your goal, I recommend turning this into a structured “**2-Year Java + Spring Boot Product Company Interview Bible**” with:

Module 1: 150 Core Java questions + answers

Module 2: 120 Spring/Spring Boot questions + internal working

Module 3: 80 JPA/Hibernate questions

Module 4: 75 REST/API questions

Module 5: 75 Microservices questions

Module 6: 75 Spring Security/JWT questions

Module 7: 50 SQL questions + queries

Module 8: 50 Multithreading questions

Module 9: 50 Kafka + Redis questions

Module 10: 50 Docker/AWS questions

Module 11: 100 DSA problems specifically for 2-year Java SDE interviews

Module 12: 50 LLD/system-design scenarios

Module 13: 100 project/resume follow-up questions

Module 14: Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions. Spring Security/JWT questions**, where we can go substantially deeper into the security-filter chain, authentication providers, JWT validation, OAuth2/OIDC, refresh tokens, stateless security, CSRF, CORS, method security, RBAC/ABAC, password encoding, session fixation, security exceptions and production security scenarios.

chatgpt response: I'll use t

I'll use the uploaded exports as the continuity source, preserve the established depth/format, verify where Module 5 ended, then continue directly into Module 6 without restarting earlier modules.

I found the exact Module 5 endpoint: Q75 is the production-grade e-commerce design, and the prior export explicitly sets Module 6 to go deeper into the security filter chain, JWT, OAuth2/OIDC, CSRF/CORS, method security, and production scenarios.

□ Module 6 — Spring Security + JWT

75 Interview Questions + Answers

2-Year Java + Spring Boot Product-Company Interview Bible

I've aligned this continuation with the attached material rather than restarting the earlier modules. The previous work establishes the sequence **Spring** → **REST** → **JPA** → **Microservices** → **Security**, and explicitly says Module 6 should go deeper into the **security filter chain, authentication providers, JWT validation, OAuth2/OIDC, refresh tokens, stateless security, CSRF, CORS, method security, RBAC/ABAC, password encoding, session fixation, security exceptions, and production scenarios**.

The established interview format also remains:

Question → **Short interview answer** → **Detailed explanation** → **Real project example** → **Follow-up** → **Tricky counter-question** → **Best answer** → **Code/diagram where relevant**.

One current-docs update is important: the Spring Security reference currently documents **7.1.0 as stable**, with 7.0.6 and 6.5.11 also listed as stable lines, so I'll use the current Spring Security architecture terminology rather than old `WebSecurityConfigurerAdapter`-style explanations. ([Home](#))

PART A — SECURITY FUNDAMENTALS

Q1–Q15

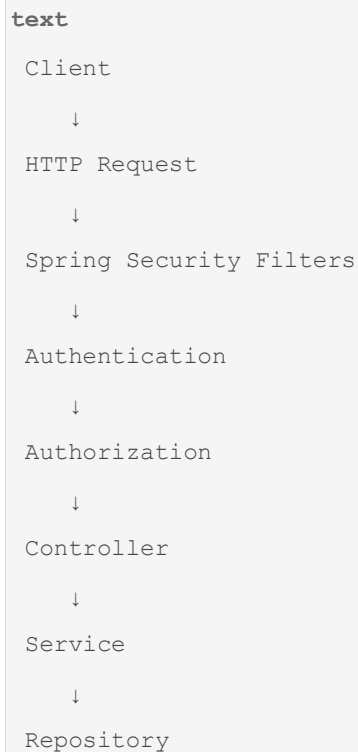
Q1. What is Spring Security? □ □ □

Short interview answer

Spring Security is a framework for securing Spring applications. It primarily provides **authentication, authorization, protection against common attacks, and security-context management.**

Detailed explanation

Think of Spring Security as a security layer around the request:



It can handle:

- username/password authentication
- JWT/Bearer-token authentication
- OAuth2/OIDC
- authorization based on roles/authorities
- CSRF protection
- security headers
- session management
- password encoding
- method-level authorization

Spring Security's servlet architecture is fundamentally filter-based. `SecurityFilterChain` determines which security filters apply to a request, with `FilterChainProxy` acting as the central entry point for Spring Security's servlet support. ([Home](#))

Real project example

For an e-commerce system:

```
text
POST /api/orders
Authorization: Bearer <JWT>
```

Security validates the token before the request reaches `OrderController`.

Follow-up

Does Spring Security authenticate every request?

Not necessarily in the same way.

For a JWT resource server, a bearer token can be processed on requests carrying the token. Public endpoints can be configured to permit unauthenticated access.

Tricky counter-question

Is Spring Security only about login?

No.

Authentication is only one part. Spring Security also handles authorization and protection against attacks.

Best answer

Spring Security is a security framework for Spring applications that provides authentication, authorization, security-context management and protection mechanisms. In a REST application, the security filter chain can authenticate a request before it reaches the controller and then enforce authorization rules.

Q2. Authentication vs Authorization? ☐ ☐ ☐ ☐

Short answer

Authentication = Who are you?

Authorization = What are you allowed to do?

Example

```
text
Username/password / JWT
    ↓
Authentication
    ↓
```

```
User identity established
```

↓

```
Authorization
```

↓

```
ROLE_ADMIN?
```

↓

```
Allow / Deny
```

Suppose:

```
text
```

```
User = Vaibhav
```

```
Role = USER
```

Then:

```
text
```

```
GET /orders
```

```
→ allowed
```

but:

```
text
```

```
DELETE /admin/users
```

```
→ denied
```

Real project example

A JWT may identify:

```
json
```

```
{
  "sub": "user123",
  "roles": ["USER"]
}
```

Authentication establishes the principal.

Authorization decides whether `USER` can execute a particular operation.

Interview trap

Can you be authenticated but unauthorized?

Yes.

A logged-in user can still receive `403 Forbidden`.

Q3. What is the Spring Security Filter Chain? □□□□□

Short answer

The Spring Security filter chain is a sequence of servlet filters that intercept HTTP requests and perform security-related processing before the request reaches application code.

Spring Security's current architecture uses `SecurityFilterChain`, managed through `FilterChainProxy`. Filters can perform authentication, authorization, exploit protection and other security tasks. ([Home](#))

Mental model

```
text
HTTP Request
  ↓
FilterChainProxy
  ↓
SecurityFilterChain
  ↓
Security Filters
  ↓
Authentication
  ↓
Authorization
  ↓
DispatcherServlet
  ↓
Controller
```

Why important?

Because interviewers often ask:

"Where exactly does JWT authentication happen?"

Not in the controller.

It happens in the security infrastructure before the request enters your controller logic.

Tricky question

Is the filter chain a single filter?

No.

It is a chain of multiple filters.

Q4. What is `SecurityFilterChain`?

Short answer

`SecurityFilterChain` defines the security filters and request-matching rules used to secure requests.

Example:

```
java
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        );

    return http.build();
}
```

What does this configuration conceptually do?

```
text
/public/**      → no authentication required
/admin/**       → ADMIN required
everything else → authenticated
```

Follow-up

Where is the chain invoked?

Spring Security's `FilterChainProxy` uses the configured security chains to determine which filters apply to the request. ([Home](#))

Q5. What is `SecurityContextHolder`? ☐ ☐ ☐

Short answer

`SecurityContextHolder` stores the current security context, which contains the current Authentication.

Spring Security's authentication architecture describes `SecurityContextHolder` as the central place where the authenticated user's security context is stored. ([Home](#))

Flow

```
text
JWT
  ↓
Authentication
  ↓
SecurityContext
  ↓
SecurityContextHolder
  ↓
Controller / Service
```

You can access it:

```
java
Authentication authentication =
    SecurityContextHolder
        .getContext()
        .getAuthentication();
```

Then:

```
java
authentication.getName();
authentication.getAuthorities();
```

Real project

Suppose the logged-in customer makes:

```
http
GET /orders
```

Your service might identify the current user from:

```
java
String username = SecurityContextHolder
```

```
.getContext ()  
.getAuthentication ()  
.getName () ;
```

Tricky question

Should business logic constantly depend directly on `SecurityContextHolder`?

Not necessarily.

For cleaner architecture, security context access can be isolated behind application-specific services or method-security mechanisms.

Q6. What is `Authentication` in Spring Security?

Short answer

`Authentication` represents the security identity and authentication information for the current request.

It contains information such as:

- principal
- credentials in applicable mechanisms
- authorities
- authenticated state

Example:

```
java  
  
Authentication auth =  
    SecurityContextHolder.getContext().getAuthentication();  
  
System.out.println(auth.getName());  
System.out.println(auth.getAuthorities());
```

Important distinction

Before successful authentication:

```
text  
Authentication request
```

After successful authentication:

text

Authenticated identity

Q7. What is `GrantedAuthority`?

Short answer

`GrantedAuthority` represents an authority granted to the authenticated principal.

Examples:

text

ROLE_ADMIN

ROLE_USER

SCOPE_orders.read

SCOPE_orders.write

Spring Security's authentication architecture explicitly uses `GrantedAuthority` for roles and scopes. ([Home](#))

Role vs authority

A role is commonly represented as:

text

ROLE_ADMIN

while a permission could be:

text

ORDER_READ

ORDER_WRITE

Interview trap

Are roles and authorities exactly the same thing?

No.

A role is a common convention built on the broader authority concept.

Q8. What is `AuthenticationManager`? ☐☐☐

Short answer

`AuthenticationManager` is the API used to perform authentication.

Spring Security's servlet authentication architecture defines `AuthenticationManager` as the API through which security filters perform authentication. The common implementation is `ProviderManager`. ([Home](#))

Flow

```
text
Security Filter
    ↓
AuthenticationManager
    ↓
AuthenticationProvider
    ↓
authenticate()
    ↓
Authentication
```

Example conceptual flow

```
java
Authentication authentication =
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            username,
            password
        )
    );
```

If successful:

```
text
Authentication
    ↓
SecurityContextHolder
```

Q9. What is `AuthenticationProvider`? ☐ ☐ ☐

Short answer

`AuthenticationProvider` performs a specific authentication mechanism.

For example:

text

```
Username/password
→ DaoAuthenticationProvider

JWT
→ JwtAuthenticationProvider

Other custom mechanisms
→ Custom AuthenticationProvider
```

Key idea

`AuthenticationManager` is the coordinator.

`AuthenticationProvider` performs the specific authentication.

Architecture

text

```
AuthenticationManager
|
+---- Provider A
|
+---- Provider B
|
+---- Provider C
```

Spring Security's standard architecture uses `ProviderManager` as the common implementation of `AuthenticationManager`, delegating authentication work to `AuthenticationProviders`. ([Home](#))

Q10. What is `UserDetailsService`? ☐ ☐ ☐

Short answer

`UserDetailsService` is an abstraction used to retrieve user information for username/password authentication.

Example:

java

```
@Service

public class CustomUserDetailsService
    implements UserDetailsService {
```

```

@Override
public UserDetails loadUserByUsername(String username) {
    // load user from DB
    return ...;
}
}

```

Flow

text

```

username + password
    ↓
AuthenticationManager
    ↓
AuthenticationProvider
    ↓
UserDetailsService
    ↓
Database
    ↓
UserDetails
    ↓
Password verification

```

Tricky question

Does `UserDetailsService` verify the password?

Not by itself.

It typically loads user information. Password comparison is handled by the authentication mechanism/provider using a password encoder.

Q11. What is `PasswordEncoder`? ☐☐☐☐

Short answer

`PasswordEncoder` hashes passwords so that applications do not store plaintext passwords.

Example:

```
java
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Then:

```
java
String hash = passwordEncoder.encode(password);
```

Verification:

```
java
passwordEncoder.matches(rawPassword, storedHash);
```

Critical interview point

You should **never** do:

```
java
password.equals(databasePassword)
```

for stored passwords.

Real project

Database:

```
text
id | username | password_hash
```

not:

```
text
password = "mypassword"
```

Tricky question

Why not SHA-256 directly?

Password hashing should use a password-oriented adaptive function with appropriate work factor and salt, rather than treating passwords like ordinary hashes.

Q12. What is BCrypt and why is it commonly used?

Short answer

BCrypt is a password-hashing algorithm designed to be deliberately computationally expensive, making brute-force attacks more difficult.

Typical:

```
java
new BCryptPasswordEncoder()
```

Important concept

Password encoding must be:

```
text
One-way
+
Salted
+
Computationally expensive
```

Tricky question

Can you decrypt a BCrypt password?

No.

You verify the supplied password against the stored hash.

Q13. What is `@PreAuthorize`? ☐ ☐ ☐

Short answer

`@PreAuthorize` provides method-level authorization using an expression.

Example:

```
java
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
}
```

Or:

```
java
@PreAuthorize("hasAuthority('ORDER_WRITE')")
```

Why useful?

URL security answers:

text

```
Can this request access /admin/**?
```

Method security answers:

text

```
Can this authenticated principal execute THIS method?
```

Interview point

This is especially useful when authorization depends on business logic.

Q14. What is the difference between URL security and method security?

URL security

java

```
.requestMatchers("/admin/**")  
.hasRole("ADMIN")
```

Method security

java

```
@PreAuthorize("hasRole('ADMIN')")
```

Architecture

text

```
HTTP request  
  ↓  
URL authorization  
  ↓  
Controller  
  ↓  
Service  
  ↓  
Method authorization
```

Why both?

URL security gives coarse-grained protection.

Method security allows finer-grained business rules.

Q15. What happens when authentication fails vs authorization fails? ☐☐☐

This is extremely important.

Authentication failure

Identity could not be established.

Typically:

```
text
401 Unauthorized
```

Authorization failure

Identity is known, but permission is insufficient.

Typically:

```
text
403 Forbidden
```

Example

No token:

```
http
GET /admin/users
```

→ 401

Authenticated USER:

```
http
DELETE /admin/users/10
```

but lacks ADMIN authority:

→ 403

Memory trick

```
text
401 → Who are you?
403 → I know who you are, but you can't do this.
```

PART B — AUTHENTICATION INTERNALS

Q16–Q30

Q16. Explain the complete username/password authentication flow.

□ □ □ □ □

text

```
Client
↓
POST /login
username/password
↓
Security Filter
↓
AuthenticationManager
↓
AuthenticationProvider
↓
UserDetailsService
↓
Database
↓
UserDetails
↓
PasswordEncoder
↓
Authentication success
↓
SecurityContext
```

Let's break it down.

Suppose:

text

```
username = rahul
password = secret
```

The authentication mechanism creates an authentication request.

Then:

```
text
AuthenticationManager
```

delegates to an appropriate provider.

The provider loads:

```
text
rahul
```

from the data source.

It obtains:

```
text
storedPasswordHash
```

and verifies the supplied password using:

```
java
passwordEncoder.matches(...)
```

On success:

```
text
Authentication
    ↓
SecurityContext
```

Interview gold

Don't answer:

"Spring checks username/password."

Say:

"The authentication request is processed by the security filter infrastructure, delegated through AuthenticationManager to an AuthenticationProvider, which loads user details, validates credentials using the configured password encoder, and places the successful Authentication into the SecurityContext."

Q17. What is `ProviderManager`?

Short answer

`ProviderManager` is the common implementation of `AuthenticationManager`.

It delegates to one or more `AuthenticationProviders`.

```
text
ProviderManager
    ↓
AuthenticationProvider 1
AuthenticationProvider 2
AuthenticationProvider 3
```

This allows different authentication mechanisms to coexist.

Spring Security's architecture identifies `ProviderManager` as the most common `AuthenticationManager` implementation. ([Home](#))

Q18. What happens if multiple `AuthenticationProviders` exist?

Suppose:

```
text
Provider A → username/password
Provider B → another authentication type
Provider C → custom token
```

`ProviderManager` delegates according to which provider can support the authentication object.

Conceptually:

```
text
Authentication
    ↓
ProviderManager
    ↓
Can Provider A support it?
    ↓
yes → authenticate
no
    ↓
Provider B
```

Tricky interview question

Why not put every authentication mechanism into one giant provider?

Because separate providers maintain clearer authentication responsibilities and integrate with Spring Security's provider architecture.

Q19. What is a custom `AuthenticationProvider`?

Sometimes your company has a custom authentication mechanism.

Example:

```
text
Employee token
  ↓
Custom AuthenticationProvider
  ↓
Internal identity system
```

You can implement:

```
java
public class CustomAuthenticationProvider
    implements AuthenticationProvider {

    @Override
    public Authentication authenticate(
        Authentication authentication) {
        ...
    }

    @Override
    public boolean supports(
        Class<?> authentication) {
        ...
    }
}
```

Real project

Suppose an enterprise uses:

```
text
EmployeeId + PIN
```

against a legacy authentication platform.

A custom authentication provider may integrate that mechanism.

Q20. What is stateless authentication? ☐☐☐☐

Short answer

Stateless authentication means the server does not need to maintain per-user login session state between requests.

Each request carries the credentials needed to authenticate it.

For JWT:

```
http
Authorization: Bearer eyJ...
```

Flow

```
text
Request 1
JWT → authenticate

Request 2
JWT → authenticate

Request 3
JWT → authenticate
```

The server does not depend on an HTTP session containing the user's authentication state.

Advantage

Good fit for:

- REST APIs
- distributed systems
- horizontally scaled services
- microservices

Tricky question

Does stateless mean no state anywhere?

No.

Databases, caches, revocation lists, token stores and other infrastructure can still hold state.

Q21. Stateful vs stateless authentication?

Stateful	Stateless
Server stores session	Token carries authentication data
Session ID usually sent	Bearer token commonly sent
Session storage required	Per-request token validation
Revocation can be straightforward	Revocation requires additional design
Traditional web apps	Common for APIs/microservices

Important nuance

Stateless JWT does **not automatically mean better**.

You choose based on:

- revocation requirements
- architecture
- clients
- security model
- operational complexity

Q22. Why is JWT popular for microservices?

Short answer

A JWT can allow a service to validate authentication information locally without calling a central session store on every request.

```
text
Client
  ↓ JWT
Order Service
  ↓ local validation

Payment Service
  ↓ local validation

Inventory Service
  ↓ local validation
```

Benefit

This can reduce the dependency on a centralized session lookup.

But...

JWT introduces difficult questions:

- token revocation
- expiration
- key rotation
- token theft
- audience validation
- refresh tokens

So:

JWT removes some server-side session state but does not remove security complexity.

Q23. What is the difference between session-based authentication and JWT authentication?

Session

```
text
Client
↓
Session ID
↓
Server session
↓
Authentication state
```

JWT

```
text
Client
↓
JWT
↓
Validate signature/claims
↓
Authentication
```

Session advantage

Revocation is often easier.

JWT advantage

Convenient for distributed APIs and stateless resource servers.

Q24. What is a Bearer Token? ☐☐☐

A bearer token means:

Whoever possesses the token can present it as the credential.

Example:

```
http
Authorization: Bearer <access-token>
```

Major risk

Bearer token theft.

If an attacker obtains a valid token, they may be able to use it until it expires or is otherwise invalidated.

Therefore:

```
text
HTTPS
+
short token lifetime
+
secure storage
+
proper audience/issuer validation
+
least privilege
```

are important.

Q25. What is a JWT?

Short answer

JWT stands for JSON Web Token.

A JWT generally has three encoded sections:

text

HEADER.PAYLOAD.SIGNATURE

Example concept

text

Header

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

Payload

```
{
  "sub": "123",
  "iss": "https://idp.example.com",
  "aud": "order-api",
  "exp": 1770000000
}
```

Signature

...

Critical interview point

JWT payload is **not automatically secret**.

It is encoded, not encrypted, unless you are specifically using an encrypted token format.

Never place secrets such as passwords in a normal signed JWT.

Q26. What are JWT claims?

Claims are pieces of information inside the token.

Common registered claims include:

text

iss → issuer

sub → subject

aud → audience

exp → expiration

nbf → not before

```
iat → issued at
jti → token identifier
```

Custom claims may also exist:

```
json
{
  "department": "FINANCE",
  "permissions": ["REPORT_READ"]
}
```

Interview trap

Should authorization trust every custom claim?

No.

Claims must be trusted only after successful token validation.

Q27. How does Spring Security validate a JWT? □□□□□

This is one of the most important questions of Module 6.

For a JWT resource server, Spring Security can use a `JwtDecoder` to decode and validate JWT bearer tokens. Current Spring documentation describes validation of the signature plus claims such as `exp`, `nbf`, and `iss`, with scopes mapped to `SCOPE_...` authorities by default. ([Home](#))

Flow

```
text
HTTP Request
Authorization: Bearer JWT
    ↓
Bearer token processing
    ↓
JwtDecoder
    ↓
Signature validation
    ↓
Claims validation
    ↓
Authentication created
    ↓
SecurityContext
```

↓
Authorization

Typical validation

```
text
Signature
iss
exp
nbf
aud (when configured)
```

Important

Authentication is not:

```
text
decode JWT → trust it
```

It is:

```
text
decode
+
cryptographic verification
+
claim validation
+
authority mapping
```

Q28. What is `issuer-uri` in Spring Security JWT configuration? ☐☐☐

Example:

```
yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.example.com
```

The issuer identifies the authorization server expected to issue the token.

Spring Security can use the issuer metadata to discover key information and configure JWT validation. ([Home](#))

Why important?

Suppose an attacker generates a token with:

```
json
{
  "iss": "https://evil.example"
}
```

Your service should not accept it merely because its signature happens to validate under some key.

Issuer validation establishes who is expected to issue the token.

Q29. What is audience validation? ☐ ☐ ☐

`aud` means:

Who is this token intended for?

Suppose:

```
json
{
  "aud": "order-api"
}
```

Then the Order Service should verify that the token is intended for it.

Spring Boot supports JWT audience configuration, and Spring Security can validate `aud`. ([Home](#))

Why important?

Imagine the same identity provider issues tokens for:

```
text
payment-api
order-api
admin-api
```

A valid token for `payment-api` should not automatically be treated as valid for `admin-api`.

Interview answer

Signature validation tells me the token was signed by a trusted issuer; audience validation additionally tells me the token was intended for my service.

Excellent answer.

Q30. What is key rotation in JWT?

A production authorization server should be able to rotate signing keys.

Suppose:

```
text
Old key → K1
New key → K2
```

New tokens may use K2.

Consumers must be able to obtain trusted public keys needed to verify tokens.

Spring's JWT resource-server support can discover and use the provider's JWK set and can automatically rotate the keys used for JWT validation as new keys become available. ([Home](#))

Interview trap

Does the resource server store the private key?

Normally no.

The authorization server keeps private signing keys.

The resource server uses trusted public keys to verify signatures.

PART C — JWT DEEP DIVE

Q31–Q45

Q31. Access token vs refresh token? ☐☐☐☐☐

Access token

Used to access APIs.

```
text
Client
↓
Access Token
↓
Order API
```

Usually short-lived.

Refresh token

Used to obtain a new access token.

text

Refresh Token

↓

Authorization Server

↓

New Access Token

Why two tokens?

You don't want the API access credential to live indefinitely.

Example:

text

Access token = 10 minutes

Refresh token = longer lifetime

Exact lifetimes depend on the security design.

Q32. Why should access tokens be short-lived?

Suppose:

text

Access token lifetime = 1 hour

If stolen, attacker may potentially use it for that period.

With:

text

Access token lifetime = 5-15 minutes

the exposure window is reduced.

But...

Short expiration alone doesn't solve everything.

You still need:

- HTTPS
- secure storage
- token validation
- refresh-token protection

- appropriate scopes
- monitoring

Q33. Should JWT access tokens contain roles?

They can.

Example:

```
json
{
  "sub": "1001",
  "roles": ["USER"]
}
```

or scopes:

```
json
{
  "scope": "orders.read orders.write"
}
```

But should you put every permission into JWT?

Not always.

Huge tokens create:

- network overhead
- stale authorization information
- difficult permission changes

A strong architecture chooses claims appropriate to the use case.

Q34. What happens if a user's role changes after JWT issuance?

Suppose:

```
text
10:00
JWT says ADMIN
```

At:

text

10:05

Admin role removed

But the JWT remains valid until expiration unless another mechanism invalidates it.

This is a major JWT trade-off.

Possible approaches:

- short-lived access tokens
- token revocation strategy
- token introspection
- authorization checks against current state
- version/revocation claims
- centralized session/token management

Interview answer

JWT authorization claims are snapshots of authorization state at token issuance. If permissions can change immediately, I need a revocation or server-side authorization strategy rather than relying solely on long-lived self-contained tokens.

Q35. What is JWT revocation?

JWT is often described as stateless, but sometimes you need:

"Immediately invalidate this token."

Possible techniques:

text

Token blacklist

JTI denylist

Short expiration

Token version

Centralized session state

Opaque token introspection

Trade-off

A blacklist introduces centralized state.

Therefore:

```
text
Statelessness
vs
Immediate revocation
```

is a design trade-off.

Q36. What is token replay attack?

Suppose attacker steals:

```
text
Bearer JWT
```

Then reuses it:

```
text
Attacker
↓
Authorization: Bearer stolen-token
↓
API
```

That's replay.

Defenses

- HTTPS
- short token expiry
- secure client storage
- sender-constrained tokens where appropriate
- refresh-token rotation strategies
- revocation
- anomaly detection

Q37. Where should JWTs be stored on the frontend?

This depends heavily on the application architecture.

The simplistic answer:

```
"Always localStorage."
```

is not a good interview answer.

Important risks

Browser storage can be exposed to malicious script in XSS scenarios.

Cookies have different characteristics and are automatically sent by browsers, which brings CSRF considerations.

Strong answer

Token storage depends on the client architecture and threat model. I would avoid casually recommending browser-accessible persistent storage for sensitive tokens and would design around XSS and CSRF risks, using secure cookie attributes and an appropriate OAuth/OIDC flow where suitable.

Q38. JWT vs opaque token?

JWT

Resource server can validate the token locally.

```
text
Token
↓
Signature/claims
↓
Local validation
```

Opaque token

Resource server generally asks the authorization server/introspection endpoint to determine token validity.

```
text
Opaque Token
↓
Introspection
↓
Authorization Server
↓
Valid?
```

Spring Security supports both JWT and opaque bearer-token approaches for resource servers. ([Home](#))

Trade-off

JWT:

text

- + local validation
- + scalable
- revocation complexity

Opaque:

text

- + centralized control
- + easier revocation model
- introspection dependency/latency

Q39. What is an OAuth2 Resource Server? ☐☐☐

A resource server is the application that hosts protected resources/APIs.

Example:

text

```
Authorization Server
      ↓ access token
Order API
      ↑
Resource Server
```

Spring Security's OAuth2 support explicitly provides Resource Server functionality for protecting APIs using JWT or opaque bearer tokens. ([Home](#))

Typical architecture:

text

```
Frontend
  ↓
Authorization Server
  ↓
Access Token
  ↓
Order API
```

Q40. What is the difference between Authorization Server and Resource Server?

Authorization Server

Issues tokens.

Example responsibility:

```
text
login
consent
token issuance
identity
```

Resource Server

Protects the API.

```
text
GET /orders
Authorization: Bearer <token>
```

and validates the token.

Simple diagram

```
text
      Authorization Server
      |
      Token
      ↓
Client -----> Resource Server
                  Order API
```

Q41. What is OAuth2? □□□□

OAuth2 is an authorization framework that allows a client to obtain limited access to protected resources.

It separates:

```
text
Client
Authorization Server
```

```
Resource Server
Resource Owner
```

Example

```
text
User
↓
Google login/authorization
↓
Authorization Server
↓
Access token
↓
Application
```

OAuth2 is about delegated authorization.

Q42. OAuth2 vs JWT?

This is a classic trap.

They are not alternatives.

OAuth2 is an authorization framework.

JWT is a token format.

You can have:

```
text
OAuth2
+
JWT access token
```

Example

```
text
OAuth2 authorization flow
↓
Access Token
↓
JWT format
```

Best interview answer

OAuth2 defines how authorization is delegated and how access tokens are obtained. JWT defines a compact token representation. An OAuth2 system may issue JWT access tokens, but OAuth2 itself is not synonymous with JWT.

Q43. What is OpenID Connect? □□□□

OpenID Connect, or OIDC, is an identity layer built on OAuth2.

OAuth2 primarily deals with authorization.

OIDC adds standardized identity information.

Example

```
text
Login with Google
      ↓
OIDC
      ↓
Identity information
      ↓
Application
```

Spring Security's OAuth2 Login support uses the Authorization Code Grant and supports OIDC when the `openid` scope is present. ([Home](#))

Q44. OAuth2 vs OIDC?

OAuth2

```
text
Can this application access this resource?
```

OIDC

```
text
Who is this user?
```

Memory trick

```
text
OAuth2 → authorization
OIDC   → authentication/identity layer
```

Interview trap

"OAuth2 is authentication."

Too simplistic and usually wrong.

Q45. Explain Authorization Code flow.

A simplified flow:

```
text
User
↓
Client Application
↓ redirect
Authorization Server
↓
Login / Consent
↓
Authorization Code
↓
Client
↓
Token Endpoint
↓
Access Token
```

For modern public clients, PKCE is an important additional protection for authorization-code flows.

Why not send username/password directly to every application?

Because centralized identity/authentication reduces credential exposure and delegates authentication to the identity provider.

PART D — OAUTH2, CSRF, CORS, SESSIONS

Q46–Q60

Q46. What is PKCE? ☐☐☐

PKCE stands for Proof Key for Code Exchange.

It protects authorization-code flows against interception of the authorization code.

Conceptually:

text

Client generates verifier

↓

Creates challenge

↓

Authorization request

↓

Authorization Server

↓

Code

↓

Token request + verifier

↓

Access Token

Interview answer

PKCE binds the authorization code exchange to the client-generated proof, reducing the risk that an intercepted authorization code can be exchanged by an attacker.

Q47. What is CSRF? ☐☐☐☐☐

CSRF = Cross-Site Request Forgery.

The attacker causes a victim's browser to send a request to a trusted site where the victim is already authenticated.

Example:

text

Victim logged into bank.example

↓

Visits attacker.example

↓

Browser automatically sends authenticated request

↓

Bank changes state

The key danger comes from browser credentials being automatically attached, such as cookies.

Q48. Does a JWT REST API need CSRF protection?

This requires nuance.

If the API uses a bearer token explicitly supplied in the `Authorization` header and does not rely on browser automatically attached authentication cookies, the classic browser-cookie CSRF scenario may not apply in the same way.

But if authentication uses cookies:

text

```
Cookie: SESSION=abc
```

CSRF becomes a significant concern.

Strong answer

CSRF protection depends on how authentication credentials are transported. Cookie-based browser authentication is vulnerable to cross-site request forgery unless protected appropriately. A header-based bearer-token API has a different threat model, so CSRF configuration should follow the actual authentication mechanism rather than blindly enabling or disabling it.

Q49. What is CORS? □□□□

CORS = Cross-Origin Resource Sharing.

It controls whether browser JavaScript from one origin may access resources from another origin.

Example:

text

Frontend:

```
https://app.example.com
```

API:

```
https://api.example.com
```

The browser may enforce CORS rules.

Important distinction

CORS is primarily a **browser enforcement mechanism**.

Postman does not normally enforce browser CORS rules.

The earlier REST module established this distinction explicitly.

Q50. What is a CORS preflight request?

For certain cross-origin requests, browser sends:

```
http
OPTIONS /orders
Origin: https://app.example.com
Access-Control-Request-Method: POST
```

Server responds with allowed origin/method/headers.

Then browser sends actual request.

Flow

```
text
Browser
↓
OPTIONS
↓
API
↓
Allowed?
↓
POST
```

Interview trap

Does every cross-origin request always cause preflight?

No.

Certain "simple" requests can be sent without a preflight.

Q51. CORS vs CSRF?

CORS	CSRF
Browser cross-origin access policy	Forged state-changing requests
Controls browser access to response	Protects against unauthorized action
Mainly browser concern	Mainly browser credential-forgery concern
Origin-based	Request-authentication/anti-forgery defense

Excellent interview answer

CORS and CSRF solve different problems. CORS controls whether browser code can access cross-origin resources, while CSRF protects authenticated applications from forged requests made using automatically supplied credentials.

Q52. What is session fixation? ☐☐☐

Session fixation occurs when an attacker causes a victim to authenticate using a session identifier known to the attacker.

Conceptually:

```
text
Attacker knows session ID
    ↓
Victim logs in using same session
    ↓
Session remains associated with authenticated user
    ↓
Attacker reuses known session ID
```

Defense

Session ID should be changed/renewed when authentication occurs.

Modern Spring Security has session-management protections for this threat.

Q53. What is session hijacking?

Session hijacking is stealing a valid session identifier and then using it.

Example:

```
text
Victim
    ↓
SESSION=abc123

Attacker steals abc123
    ↓
Application
```

Protection

- HTTPS

- Secure cookies
- HttpOnly cookies
- SameSite policy
- short session lifetime
- session invalidation
- secure token handling

Q54. Explain `HttpOnly`, `Secure`, and `SameSite`.

`HttpOnly`

Prevents JavaScript from accessing the cookie directly.

```
text
document.cookie
```

cannot read an HttpOnly cookie.

`Secure`

Cookie should be sent over secure HTTPS connections.

`SameSite`

Controls cross-site cookie sending behavior.

Common values:

```
text
Strict
Lax
None
```

Interview answer

These flags are particularly important for browser-based session/cookie authentication.

Q55. What is the purpose of `SecurityContextPersistence`?

At a high level, Spring Security needs to load and save security context around request processing.

Conceptually:

```
text
Request starts
```

```
↓
SecurityContext available
↓
Security filters
↓
Controller
↓
Request ends
↓
Context handled/cleared
```

Spring Security's architecture documentation highlights `FilterChainProxy` as central and notes that it clears the `SecurityContext` to avoid memory leaks. ([Home](#))

Important

Don't rely on the security context hanging around globally forever.

Its lifecycle is tied to security processing.

Q56. What is the difference between ``permitAll()`` and ignoring security?

This is an excellent tricky question.

``permitAll()``

The request still passes through security infrastructure but is allowed by authorization rules.

Example:

```
java
.requestMatchers("/login").permitAll()
```

Ignoring

Can bypass portions of Spring Security entirely for matching requests/resources.

Strong answer

permitAll() means the request is handled by the security chain but authorization allows it. Ignoring is a different concept where matching resources may bypass the security chain, so I would use it carefully.

Q57. Why should static resources sometimes be ignored?

For genuine static resources, such as:

```
text
/css/**
/js/**
/images/**
```

you may configure security appropriately to avoid unnecessary processing.

But don't casually ignore API endpoints.

Q58. What is CSRF token validation?

For cookie-authenticated applications, server generates or otherwise establishes an anti-CSRF token.

Client must submit the token with state-changing requests.

Conceptually:

```
text
GET form
↓
CSRF token
↓
POST + token
↓
Server validates
```

Without the correct token:

```
text
403
```

Why does this work?

The attacker may be able to trigger a request, but should not be able to obtain the legitimate anti-CSRF token because of browser same-origin protections.

Q59. What are Spring Security exception handlers?

Two important mechanisms:

`AuthenticationEntryPoint`

Used when authentication is required or fails.

Typically results in:

```
text
401
```

`AccessDeniedHandler`

Used when an authenticated user lacks sufficient permission.

Typically:

```
text
403
```

Mental model

```
text
Not authenticated
    ↓
AuthenticationEntryPoint
    ↓
401

Authenticated
but insufficient permission
    ↓
AccessDeniedHandler
    ↓
403
```

Q60. Why should Spring Security errors not expose internal details?

Suppose an API returns:

```
json
{
  "error": "User admin123 does not exist in LDAP server XYZ..."
}
```

That's dangerous information leakage.

Security APIs should return controlled error structures.

Example:

```
json
{
  "timestamp": "...",
  "status": 401,
  "error": "Unauthorized",
  "message": "Authentication required",
  "path": "/api/orders"
}
```

Avoid exposing:

- database details
- stack traces
- internal hostnames
- secret configuration
- credential-processing errors

PART E — AUTHORIZATION, RBAC, ABAC + PRODUCTION

Q61–Q75

Q61. What is RBAC? ☐☐☐☐

RBAC = Role-Based Access Control.

Example:

```
text
USER
ADMIN
MANAGER
SUPPORT
```

Authorization depends on assigned roles.

```
java
@PreAuthorize("hasRole('ADMIN')")
```

Example

text

```
ADMIN → delete user  
USER  → read own profile
```

Advantage

Simple and easy to understand.

Problem

Large systems can accumulate too many roles.

Q62. What is ABAC?

ABAC = Attribute-Based Access Control.

Instead of only checking:

text

```
role == ADMIN
```

authorization can consider attributes such as:

text

```
user.department  
resource.owner  
resource.department  
request.time  
location  
operation
```

Example

text

```
Employee can read document if:  
  
employee.department == document.department  
AND  
employee.permission == "READ"
```

RBAC

text

```
ADMIN → allowed
```

ABAC

text

```
user.department == resource.department  
AND user.permission contains READ
```

Product-company point

RBAC is simpler.

ABAC is more expressive.

Q63. How do you implement resource ownership authorization?

Suppose:

text

```
GET /orders/100
```

User A should access only orders belonging to User A.

A role check isn't sufficient.

You need:

text

```
authenticated user ID  
    ↓  
load order  
    ↓  
order.customerId == authenticatedUserId?  
    ↓  
allow/deny
```

Important distinction

This is more granular than:

java

```
hasRole("USER")
```

because all normal users may have the same role but not the same resource ownership.

Q64. `hasRole()` vs `hasAuthority()`?

``hasRole``

Commonly used with role convention:

```
java
hasRole("ADMIN")
```

Spring Security typically treats that in terms of:

```
text
ROLE_ADMIN
```

``hasAuthority``

Uses the exact authority:

```
java
hasAuthority("ORDER_READ")
```

Example

```
java
@PreAuthorize("hasRole('ADMIN')")
```

versus:

```
java
@PreAuthorize("hasAuthority('ORDER_READ')")
```

Interview trap

Don't confuse the string:

```
text
ADMIN
```

with the underlying conventional authority:

```
text
ROLE_ADMIN
```

Q65. How does JWT scope become a Spring authority?

For JWT resource-server support, Spring Security maps scopes to authorities with the prefix:

text

```
SCOPE_
```

So:

json

```
{
  "scope": "orders.read orders.write"
}
```

can conceptually become:

text

```
SCOPE_orders.read
SCOPE_orders.write
```

Then:

java

```
.hasAuthority("SCOPE_orders.read")
```

can be used.

This mapping is documented in Spring Security's JWT resource-server behavior. ([Home](#))

Q66. How do you configure a JWT resource server in Spring Boot? ☐☐☐☐

Typical configuration:

yaml

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.example.com
```

And then:

java

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
```

```

    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**")
                .hasAuthority("SCOPE_admin")
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 ->
            oauth2.jwt());

    return http.build();
}

```

Spring Boot can auto-configure an OAuth2 Resource Server when the appropriate dependency is present and JWT configuration is supplied. ([Home](#))

Q67. What is the current replacement for `WebSecurityConfigurerAdapter`?

This is a particularly important modern Spring interview question.

Older Spring Security code frequently used:

```

java
extends WebSecurityConfigurerAdapter

```

Modern Spring Security configuration generally uses beans such as:

```

java
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)

```

Example:

```

java
@Bean
SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

    http
        .authorizeHttpRequests(auth -> auth

```

```

        .anyRequest()
        .authenticated()
    );

    return http.build();
}

```

Interview answer

In modern Spring Security, we configure security using SecurityFilterChain beans rather than relying on the older WebSecurityConfigurerAdapter approach.

This avoids answering from outdated Spring Security tutorials.

Q68. How would you design security for a microservices architecture?

□ □ □ □ □

Suppose:

```

text
Client
  ↓
API Gateway
  ↓
Order Service
  ↓
Payment Service
  ↓
Inventory Service

```

A robust architecture might look like:

```

text
                                Identity Provider
                                |
                                JWT/OIDC
                                ↓
Client → API Gateway → Services
    |               |
    |               +→ Order
    |               +→ Payment
    |               +→ Inventory

```

Security layers

text

HTTPS

↓

Gateway authentication

↓

JWT validation

↓

Service-level validation

↓

Authorization

↓

Audit logging

Important

Do not blindly assume:

"Gateway validates JWT, therefore internal services don't need security."

A compromised internal network or direct service access can otherwise bypass assumptions.

Q69. Should every microservice validate the JWT?

There are architectural choices.

Option A

Gateway validates, downstream trusts gateway.

Option B

Gateway validates + each service validates.

Option B is stronger isolation but can add complexity.

Strong interview answer

In a security-sensitive microservice architecture, I generally prefer services to be able to independently enforce their own authorization boundaries rather than relying solely on the gateway. The exact design depends on the trust boundary, network architecture and operational requirements.

Excellent product-company answer.

Q70. What is defense in depth in application security?

Never rely on one protection.

Example:

```
text
HTTPS
↓
API Gateway
↓
JWT validation
↓
Authorization
↓
Input validation
↓
Database access controls
↓
Audit logging
↓
Monitoring
```

If one layer fails, other controls remain.

Example

Even if a user has a valid JWT:

```
text
valid token
≠
permission to modify any resource
```

Q71. How do you secure passwords in a Spring Boot application? ☐☐☐☐

Correct flow

```
text
Registration
↓
Raw password
```

```
↓
PasswordEncoder
↓
Hash
↓
DB

Login
↓
Raw password
↓
matches(raw, storedHash)
↓
Success/failure
```

Never:

```
text
store plaintext password
```

Never:

```
text
decrypt password
```

because passwords should be stored as one-way password hashes.

Q72. Production scenario: An attacker steals a JWT. What do you do?

☐ ☐ ☐ ☐ ☐

Don't answer:

"Delete the JWT."

A stolen bearer token is already in the attacker's possession.

Immediate response

```
text
1. Revoke/invalidate where possible
2. Reduce token lifetime
3. Rotate relevant credentials/keys if compromise warrants it
4. Investigate access logs
```

5. Identify token reuse
6. Force reauthentication where necessary
7. Review refresh tokens
8. Patch root cause

Investigate

text

Which token?
When issued?
Which IP/device?
Which APIs?
How many requests?
What data accessed?

Long-term

text

short-lived access tokens
+
secure storage
+
refresh token protection
+
monitoring
+
revocation strategy

Q73. Production scenario: JWT validation suddenly fails in every service.

What do you investigate?

Step 1 — Token itself

text

exp
iss
aud
alg
signature

Step 2 — Identity provider

text

Is it available?

Step 3 — JWK endpoint

text

Are keys available?

Did key rotation happen?

Step 4 — Clock synchronization

Check:

text

exp

nbf

iat

Clock skew can produce failures.

Spring Security provides timestamp validation, including configurable clock-skew handling. ([Home](#))

Step 5 — Configuration

text

issuer-uri

audiences

JWK URI

environment variables

secrets

Excellent interview answer

I would distinguish token-format problems from trust-configuration problems, then verify issuer, audience, signature keys, expiration, clock synchronization and identity-provider/JWK availability before changing application code.

Q74. Production scenario: Users randomly get 401 after deployment. What do you investigate? ☐ ☐ ☐ ☐ ☐

Use systematic debugging.

text

Client

↓

```
Gateway?  
↓  
Token?  
↓  
Security Filter?  
↓  
JwtDecoder?  
↓  
Issuer?  
↓  
Audience?  
↓  
JWK?  
↓  
Clock?
```

Check

1. Token expiration

```
text  
exp
```

2. Issuer

```
text  
iss
```

3. Audience

```
text  
aud
```

4. Key rotation

Old services may have stale keys/configuration.

5. Environment mismatch

Example:

```
text  
DEV issuer
```

accidentally deployed to:

```
text
PROD
```

6. Authorization header

```
http
Authorization: Bearer <token>
```

7. Gateway behavior

Did the gateway strip or rewrite the header?

Important

Don't immediately change:

```
java
permitAll()
```

just to make the problem disappear.

That's a dangerous production debugging practice.

Q75. □□□□□ FINAL MASTER QUESTION

Design a Production-Grade Spring Boot Security Architecture

This is the Module 6 equivalent of the final master questions from the previous modules.

The earlier modules built toward an architecture where:

```
text
Client
↓
HTTPS
↓
API Gateway
↓
Microservices
↓
Database
```

and emphasized failure handling, distributed transactions, idempotency, observability and production reasoning.

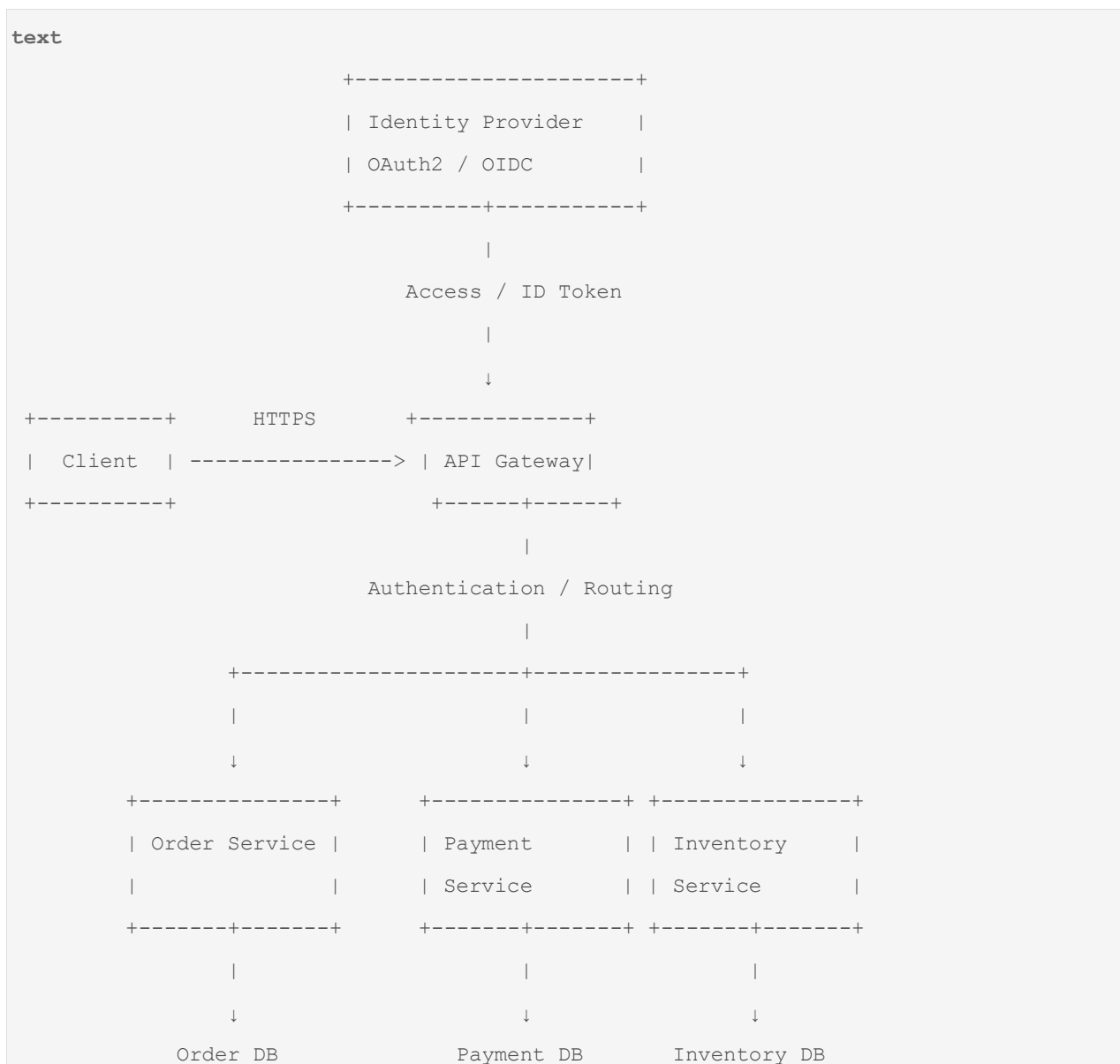
Now add security.

Interviewer asks:

"Design security for a production e-commerce microservices application. Explain authentication, authorization, JWT, API Gateway, service-level security, roles, scopes, refresh tokens, failure handling and security monitoring."

Best interview answer

I would design it like this:



Step 1 — Authentication

Use an external identity provider with OAuth2/OIDC.

```
text
User
↓
Identity Provider
↓
Access Token
↓
API
```

For login use cases, Spring Security's OAuth2 Login supports OAuth2/OIDC and uses the Authorization Code flow; OIDC is indicated by the `openid` scope. ([Home](#))

Step 2 — Access token

Use a short-lived access token.

For APIs:

```
http
Authorization: Bearer <JWT>
```

The resource server validates:

```
text
signature
iss
exp
nbf
aud
```

as appropriate.

Spring's current JWT resource-server documentation describes JWT signature and claim validation and scope-to-authority mapping. ([Home](#))

Step 3 — Gateway

Gateway performs:

```
text
TLS termination
routing
rate limiting
request filtering
```

```
observability
possibly authentication
```

But don't make it the only security boundary.

Step 4 — Microservice security

Each important service should understand its own authorization boundary.

For example:

```
text
Order Service
  |
  +-- orders.read
  +-- orders.write

Payment Service
  |
  +-- payments.read
  +-- payments.write
```

Step 5 — Authorization

Use:

```
text
RBAC
+
scopes/permissions
+
resource ownership
```

Example:

```
java
@PreAuthorize("hasAuthority('SCOPE_orders.write')")
public void createOrder(...) {
}
```

For resource ownership:

```
text
Authenticated user
```

```
↓  
Order.customerId  
↓  
must match
```

Step 6 — JWT lifecycle

```
text  
  
Short-lived Access Token  
+  
Protected Refresh Token  
↓  
New Access Token
```

Don't make access tokens unnecessarily long-lived.

Step 7 — Password security

If your system itself manages passwords:

```
text  
  
Password  
↓  
PasswordEncoder  
↓  
BCrypt/appropriate password hash  
↓  
Database
```

Never store plaintext passwords.

Step 8 — Browser security

For browser applications:

```
text  
  
HTTPS  
+  
CSRF strategy  
+  
CORS policy  
+  
Secure cookies where used
```

```
+  
HttpOnly  
+  
SameSite
```

CORS and CSRF must not be treated as the same problem.

Step 9 — Failure handling

Suppose identity provider/JWK infrastructure is unavailable.

You need to know whether already-configured JWT verification can continue.

Spring Security's JWT resource-server support can discover signing keys and validate tokens according to issuer configuration; the documented design also avoids coupling startup to authorization-server availability in the normal discovery setup. ([Home](#))

Production architecture therefore needs:

```
text  
timeout  
retry where appropriate  
circuit breaking for remote dependencies  
cached signing keys  
monitoring  
alerting
```

Step 10 — Observability

Track:

```
text  
401 count  
403 count  
JWT validation failures  
expired-token rate  
invalid issuer  
invalid audience  
JWK failures  
authorization denials  
suspicious token reuse  
login failures  
refresh-token failures
```

And use:

text

Logs
Metrics
Distributed tracing
Correlation IDs
Alerts
Audit logs

This directly follows the production-observability mindset established in the previous Microservices module.

□ Final Security Architecture Mental Model

Remember this chain for interviews:

text

CLIENT
↓
HTTPS
↓
OAuth2 / OIDC
↓
ACCESS TOKEN
↓
API GATEWAY
↓
SPRING SECURITY FILTER CHAIN
↓
BEARER TOKEN VALIDATION
↓
JwtDecoder
↓
Signature + Claims Validation
↓
Authentication
↓
SecurityContext
↓

```

Authorization
  ↓
RBAC / Authorities / Scopes / Ownership
  ↓
CONTROLLER
  ↓
SERVICE
  ↓
DATABASE

```

And around everything:

text

SECURITY CONTROLS
HTTPS
CSRF
CORS
Rate Limiting
Password Encoding
Secure Sessions/Cookies
Secrets Management
Audit Logging
Monitoring
Key Rotation
Token Revocation

☐ Module 6 — 75-question priority list

For a **2-year Java + Spring Boot product-company interview**, don't treat all 75 questions equally.

☐ Must-master

Q1 Spring Security **Q2** Authentication vs Authorization **Q3** Security Filter Chain **Q5** SecurityContextHolder **Q8** AuthenticationManager **Q9** AuthenticationProvider **Q10** UserDetailsService **Q11** PasswordEncoder **Q13** @PreAuthorize **Q15** 401 vs 403 **Q16** Username/password flow **Q20** Stateless authentication **Q22** JWT in microservices **Q27** JWT validation **Q28** issuer-uri **Q29** audience **Q31** access vs refresh token **Q32** short-lived access tokens **Q34** role change after JWT issuance **Q35** JWT revocation **Q38** JWT vs opaque token **Q39** Resource Server **Q41** OAuth2 **Q42** OAuth2 vs JWT **Q43** OIDC **Q46** PKCE **Q47** CSRF **Q48** CSRF with JWT

Q49 CORS **Q51** CORS vs CSRF **Q59** AuthenticationEntryPoint vs AccessDeniedHandler **Q61** RBAC **Q62** ABAC **Q65** JWT scopes → authorities **Q66** Spring Boot JWT configuration **Q67** modern SecurityFilterChain configuration **Q68** microservices security architecture **Q69** service-level JWT validation **Q71** password security **Q72** stolen JWT incident **Q73** JWT validation failure **Q74** random 401 production incident **Q75** complete production security design

☐ The interviewer follow-up chain you should be able to handle

Don't memorize:

"JWT is stateless."

Instead be capable of this:

text

What is JWT?

↓

How is it validated?

↓

Where is it validated?

↓

Which Spring Security filter handles it?

↓

What is AuthenticationManager?

↓

What is AuthenticationProvider?

↓

How does JwtDecoder work?

↓

How are issuer and audience validated?

↓

How are scopes converted to authorities?

↓

How do you revoke a JWT?

↓

What happens when keys rotate?

↓

What if a JWT is stolen?

↓

What if the identity provider is unavailable?

↓

How do you secure microservices?

```
↓  
How do you handle authorization?  
  
↓  
How do you monitor security failures?
```

That is the same **follow-up-depth approach** the earlier modules deliberately established instead of stopping at annotation definitions.

Current Spring Security references used for the updated portions

The current official Spring Security documentation describes the filter-based servlet architecture and `SecurityFilterChain`; OAuth2 support covers Resource Server, Client and Authorization Server roles; JWT resource-server support uses `JwtDecoder`/JWK-based validation; and OAuth2 Login supports Authorization Code + OIDC. ([Home](#))

Module 6 is now complete: 75 questions.